Project 3: Campus Compass

100 Points Possible

4/22/2026

Attempt 1

In Progress

NEXT UP: Submit Assignment

Add Comment

3/25/2026 to 4/25/2026

Details

Module 11

Project 3: Campus Compass

FAQ

FAQ Document (https://docs.google.com/document/d/1KYQgdEeoKjxQr2GiSrU6vsKDAF tab=t.0)

The course staff will maintain an active FAQ Google document to answer your questions. Post your questions in Slack, but we will answer them in this document and send you the link.

Test Environment Template

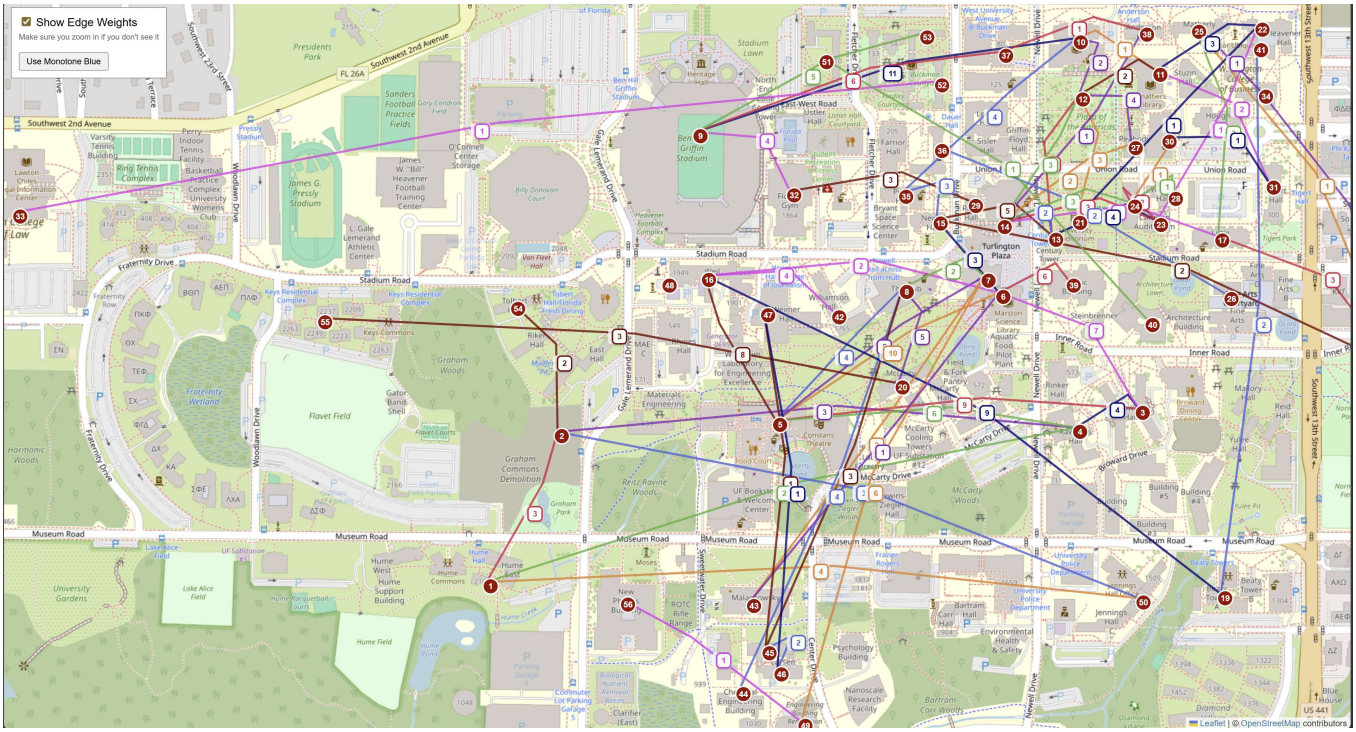
Project 3 Template (https://github.com/COP3530/P3-Catch-Template)

Like for Project 1, this template sets up an environment with Catch for use in developing and testing your code. It works with VSCode and CLion, and can be used remotely in a GitHub CodeSpace. More details are in the link.

Problem Statement

The University of Florida plans to develop an application to help students access key information about their classes, residences, and study locations. Due to campus construction and major events such as football games or guest lectures, some roads and sidewalks may be

closed. Additionally, shorter routes may become congested, making longer paths with less traffic faster to travel. For the sake of this project, all the numbers between nodes are random and not accurate to distance.



Visualizer Link: [cop3530.github.io (https://cop3530.github.io/)](https://cop3530.github.io/)

In this project, you are tasked with developing a C++ program to manage on-campus roads and sidewalks connecting student residences and classrooms. The application should support functions for **student and logistic management** such as adding and removing students, managing their classes, opening and closing roads, and finding the shortest routes from residences or study zones to classes - even accounting for closures due to construction. Your responsibilities include:

- Designing the interfaces, modules, and functions needed to handle these commands.
- Loading and managing campus data accurately.
- Parsing and validating user input and commands.
- Implementing a main function that processes commands and calls the appropriate methods.
- Testing the program to ensure it works correctly within all given constraints.

Functionality

You are responsible for organizing your implementation files (.h and .cpp). Your project must include a **main.cpp** file that:

1. Loads the data used by your program, and

2. Implements a parser that reads user input from standard input (using `std::cin` or `std::getline`), executes the corresponding commands, and prints the results to standard output using `std::cout`.

1. Load data

The main.cpp must first load the data from two CSV files with the following formats. **Download the data here: [data.zip (https://ufl.instructure.com/courses/551677/files/103073661?wrap=1)](https://ufl.instructure.com/courses/551677/files/103073661/download?download_frd=1) (https://ufl.instructure.com/courses/551677/files/103073661/download?download_frd=1)**

- The first is "edges.csv" which is an edge list representing the time it takes between two location ids. Location ids can represent classrooms, residences, or landmarks. Names are associated with the IDs.

LocationID_1	LocationID_2	Name_1	Name_2	Time
1	2	Hume Hall	Graham Hall	12
1	6	Hume Hall	Rawlings Hall	23

- The second is "classes.csv" which is a map between a class code as well as the classes' respective location ID. We have also included the start and end times for the classes; **this information is only required for the extra credit for this project.**

ClassCode	LocationID	Start Time (HH:MM)	End Time (HH:MM)
COP3530	4	10:40	11:30
CDA3101	2	11:30	1:40

Note:

- Here are some helpful links to help you load CSVs in C++:
  - **Reading CSV Files in C++: How To Guide | by ryan | Medium (https://medium.com/@ryan_forrester_/reading-csv-files-in-c-how-to-guide-35030eb378ad)**
  - **CSV File Management Using C++ - GeeksforGeeks (https://www.geeksforgeeks.org/cpp/csv-file-management-using-c/)**

- For the autograder to work properly you must structure your code like this:

```
your_root_dir/src/main.cpp (and other .h / .cpp files)

your_root_dir/data/edges.csv
your_root_dir/data/classes.csv
```

2. Command Parsing and Class Representation:

In this project, you will implement parsing logic that reads user input from standard input (using `std::cin` or `std::getline`), executes the corresponding commands, and prints the results to standard output (using `std::cout`).

These commands should operate on your data structure, represented as a `Graph` class, along with any additional data structures needed to manage student information or related data.

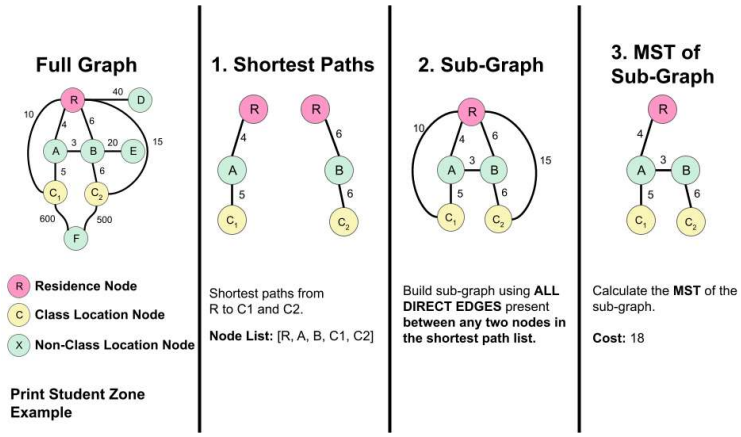
You may use helper functions as needed, but your implementation must provide full functionality for all required commands.

Command	Function
insert STUDENT_NAME STUDENT_ID RESIDENCE_LOCATION_ID N CLASSCODE_1 CLASSCODE_2 … CLASSCODE_N	• Adds a student to the class(es) with the specified name: STUDENT_NAME, STUDENT_ID, RESIDENCE_LOCATION_ID, and N classes with Class Codes: CLASSCODE_1 CLASSCODE_2 … CLASSCODE_N • The *STUDENT_ID* must be unique. • The *STUDENT_ID* and *STUDENT_NAME* must satisfy the constraints stated below. • There must be N classes after reading N; otherwise, the entry is invalid. • Also, prints "**successful**" if insertion is successful and prints "**unsuccessful**" otherwise. • *NAME* identifier will be separated by quotes for parsing, e.g., "Josh Smith." • CLASSCODE identifiers will always be a single word, e.g., COP3530
remove STUDENT_ID	• Find and remove a specific student with UFID number: *STUDENT_ID* from the data structure. • If deletion is successful, print "**successful**." • If the *STUDENT_ID* does not exist, print "**unsuccessful**."

Command	Function
dropClass STUDENT_ID CLASSCODE	• Find and drop the class, *CLASSCODE*, for a given student. • **Note:** If we remove a CLASSCODE from a student with only that CLASSCODE, the student has 0 classes and should be dropped from the data structure. ○ Still prints "**successful**" in this case • Fails if a student with *STUDENT_ID* does not exist or if *CLASSCODE* does not exist. • Fails if a student with *STUDENT_ID* does not have *CLASSCODE* • prints "**successful**" if dropping is successful and prints "**unsuccessful**" otherwise.
replaceClass STUDENT_ID CLASSCODE_1 CLASSCODE_2	• Find a student with UFID number: *ID* and replace *CLASSCODE_1* with *CLASSCODE_2* • Should fail if the student does not exist, does not have *CLASSCODE_1*, or already has *CLASSCODE_2*; or if there is no class with code *CLASSCODE_2* • prints "**successful**" if the replacement is successful and prints "**unsuccessful**" otherwise.
removeClass CLASSCODE	• Removes a class CLASSCODE from the schedule for all students. • **Note:** If we remove a CLASSCODE from a student with only that CLASSCODE and the student has 0 classes, then the student should be dropped from the data structure. • Prints the number of students the class was dropped from. ○ E.g., if 4 students were enrolled in COP3530 then "removeClass COP3530" would print "4". • Prints "**unsuccessful"** if the CLASSCODE is in an invalid format, the CLASSCODE does not exist, or no students have a given CLASSCODE.
toggleEdgesClosure N LOCATION_ID_X LOCATION_ID_Y ... LOCATION_ID_A LOCATION_ID_B	• Toggles the closure (soft delete) of *N* different edges. • **Note:** These edges are **not** being removed from the graph. It is simply closed until toggled back on. • Consecutive *LOCATION_IDs* will correspond to the same edges in this command: ○ E.g., "toggleEdgesClosure 2 5 6 9 10" would close 2 edges, the edge between 5 to 6 and 9 to 10. • If a given edge is already closed, this should open it, and if a given edge is open, this should close it. • You will always be given valid edges in this method.

Command	Function
	• prints "**successful**" on completion.
checkEdgeStatus LOCATION_ID_X LOCATION_ID_Y	• Checks whether a given edge is open, closed, or does not exist. • Prints "**open**" "**closed**" or "**DNE**" accordingly.
isConnected LOCATION_ID_1 LOCATION_ID_2	• Checks if it's possible to travel between *LOCATION_ID_1* and *LOCATION_ID_2* using any of the available edges. • Prints "**successful**" if there is a path and "**unsuccessful**" if not.
printShortestEdges ID	• Prints the shortest walking time from a student's residence to each of their classes using only currently available edges. • You will be given a valid student *ID* • **Note:** This does not take into account the start/end time of the classes. Just the time it takes to get to them using available edges. • Output should include the route for each class: Time For Shortest Edges: [Student Name] [ClassCode1]: [Time] [ClassCode2]: [Time] • If a class is currently unable to be reached, then total time should equal -1. Time For Shortest Edges: Manav Sanghvi COP3530: -1 • The classes should be sorted in lexographical order. (E.g., COP3502 would come before COP3503).

printStudentZone ID	• A student's "zone" is the minimum-cost set of edges required to connect their residence and all their classes' locations, based on their shortest-path routes.
	• You will be given a valid student *ID*
	• To calculate this, you must:
	  ○ First: find the shortest path from the student's residence to each of their classes using only currently accessible edges (**Note:** this is the same functionality as your **printShortestEdges** command).
	  ○ Second, create a sub-graph that contains all the vertices from all of those paths. Remaining edges must connect two nodes within this set of vertices.

Command	Function
	◦ Third, get the minimum spanning tree of this sub-graph. This is the “Student Zone.” • The output should be the total cost of this MST. • We will only test this on a valid subgraph (i.e., we will not test this when a student has no reachable classes). • Output Format: ◦ Student Zone Cost For [Student Name]: X • Download the image below for a visual explanation. 
Extra Credit: **verifySchedule ID**	• A student's schedule is only feasible if they can physically get from one class to the next in the allotted time. ◦ For example, if COP3530 is from 10:40-11:30 and CDA3101 is from 11:45-12:35, but the shortest path from COP3530 to CDA3101 takes longer than 15 minutes, then the schedule is not feasible. ◦ **Note:** the shortest path must use only available edges. • If the Time Gap between the classes is longer than or equal to the shortest path, we output “**successful**” • If the Time Gap between the classes is shorter than the shortest path, we print “**unsuccessful**” • In the output, the classes should be organized from the earliest start time to the latest start time, and you are checking the Time Gap between consecutive classes. • If you cannot make it from ClassCode1 to ClassCode2, still calculate for ClassCode2 to ClassCode3. • If a student has only 1 class, print “**unsuccessful**” otherwise, print: Schedule Check for [Student Name]: [ClassCode1] - [ClassCode2]: **successful** [ClassCode2] - [ClassCode3]: **unsuccessful**

Graph Data Structure

To receive full credit for this project, you must attempt to represent the given data in a Graph data structure/object class used in your main program. Additionally, this graph should:

- Make appropriate use of public and private methods.

Testing Constraints

We will ensure we test within these limits:

- 1 <= *No. of Commands* <= *1000*
- 6 <= *Length of a command* <= *1000*
- A command will always run for a single line and never contain new line characters ('\n'), except at the end.

Data Validation and Design Requirements

You need to ensure these on your end:

Data

NOTE: You need to validate the following and print “unsuccessful” if data or commands are invalid and move to the following command.

- UFIDs are strictly eight digits long and must be unique.
- Names must include only alphabets from [a-z, A-Z, and spaces]
- A students total classes must be between 1 to 6 (inclusive)
- Class Codes must have 3 capital letters [A-Z] followed by 4 digits [0-9]
- Any invalid or misspelled commands must be ignored with an “unsuccessful” message followed by executing the next command.

Design/Implementation

- You must develop the class from scratch
- You must use C++17 and ensure your code runs public test cases on Gradescope

Sample Input/Output

Input

```
7
insert "Brandon" 45679999 20 2 COP3530 MAC2311
```

```
insert "Brian" 35459999 21 3 COP3530 CDA3101 MAC2311
insert "Briana" 87879999 22 3 CDA3101 MAC2311 EEL3701
removeClass COP3530
remove 87879999
removeClass CDA3101
printShortestEdges 35459999
```

NOTE: Line 1 denotes the number of lines or the total number of commands that follow.

Output

```
successful
successful
successful
2
successful
1
Time For Shortest Edges: Brian
MAC2311: 11
```

⌨ Testing

Test your code on the Starter template and/or Gradescope. You have five available test cases, and you can submit them any number of times.

In addition to the five public test cases, your submission will be tested after the due date with ten additional test cases.

In this project, you must **create your test cases.**

Visualization

With this much data, creating test cases that accurately match the edge cases of your desired commands can be difficult. We have provided a visualizer tool that maps all the edges and nodes over a map of campus, to help you understand the structure of the dataset. This matches the “edges.csv” file that you are given with the project.

Visualizer Link: [cop3530.github.io (https://cop3530.github.io/)](https://cop3530.github.io/)

☑ Guidelines & Grading

Grading

Implementation Documentation Code Style and Design Unit Tests

Implementation [75 points]

Your code will be tested on 15 test cases, each worth 5 points:

- 5 publicly available test cases are part of Gradescope.
- The course staff will add 10 test cases during grading.
- There will also be **one additional** test case testing the verifySchedule command for extra credit.

Documentation [12 Points]

Submit a document addressing these prompts:

- Written documentation:
  - **Note** that the methods here refer to the methods you call for the respective commands. You don't need to analyze the time complexities of helper methods, constructors, etc. However, you must account for helper methods' time complexities when you analyze a command that calls those helper functions. [10 points]
  - If you implemented the extra credit method, please discuss your extra credit implementation as well.
  - What is the time complexity of each method (**corresponding to a command**) in your implementation? Reflect on the worst-case time complexity represented in Big O notation.
  - What did you learn from this assignment, and what would you do differently if you had to start over? [1 points]
  - Write 2-4 sentences on the project difficulty, intuition, and your overall thoughts on the project. [1 points]

Code Style and Design [8 Points]

The eight points for code style and design are divided into:

- Four points for the following design decisions and code modularity:
  - appropriate modularity and proper memory management.
  - the client (your int main() method) should not have objects directly interacting with memory - or a well-defined API for your class.
- One point for appropriate comments.
- One point for consistent whitespace mechanism.

- Two points for consistent naming conventions.

Unit tests [5 points]

Submit a file containing the following four tests using the Catch Framework:

- Test at least five incorrect commands. Ex: insert "A11y" 45679999 1 1 COP3530 [1 points]
- Test at least three edge cases for various functions. Ex: removing a student that doesn't exist [1 points]
- Test your dropClass, removeClass, remove, and replaceClass commands [1 point]
- Test your printShortestEdges command in a case where a student can reach a class, then one or more edges turn off, and then they cannot reach that class. (**Hint:** the visualizer tool can help with this a lot). [2 points]

Submission

You are required to upload on Gradescope the following:

- at least ***one*** .cpp file with your implementation of the entire project, including the main. If you use header files or more than one .cpp file, upload all of them together on Gradescope. Feel free to choose the names for your header and .cpp files. **Make sure at least one .cpp file contains the main() method.**
- one pdf file that has your documentation and your name on the front page of the pdf. Name this pdf as ***Report.pdf***.
- one **test.cpp** file with your five test cases using the **Catch2 framework** and a screenshot of the passing tests titled **Test**. **Make sure to comment out your entire test.cpp file.**

> Failure to follow these instructions will lead to a **10% non-negotiable penalty**.

```
Project Authors: Manav Sanghvi, Justin Brown, Daniel Park, and Amanpreet Kapoor
Version: 1.1
Last Modified: 3/13/2026

Last Edited by: Manav Sanghvi
Change Log: Updated output for errors and formatting for various functions.
```

Return to Module (https://ufl.instructure.com/courses/551677/pages/module-4-trees)

COP3530-Sp26 | Spring 2026

Course ID: 1221387

Name	Status	Released	Due (El
Project 3: Campus Compass	No Submission	Mar 26 at 12:00PM	2 weeks, 10 hou Apr 22 at 1 Late Due Date: Apr 25 at 1
Project 1 Resubmission	70.0 / 70.0	Feb 22 at 12:00AM	Mar 16 at 1
Project 1: Gator AVL Project	57.89 / 95.0	Jan 15 at 12:00AM	Feb 20 at 1 Late Due Date: Feb 23 at 1
Project 1 Testing Check-In	4.5 / 5.0	Jan 16 at 12:00AM	Feb 13 at 1 Late Due Date: Feb 16 at 1
Programming Quiz 4 - Trees and Black Box Testing	92.67 / 100.0	Feb 02 at 12:00AM	Feb 08 at 1